

Computer Graphics

Rasterisation - Based Rendering Pipeline

- | | |
|-----------------------------------|----------------------------------|
| 1. Object Transformation |] Programmable - Vertex Shader |
| 2. Perspective Projection | |
| 3. Clipping |] Fixed (hardware) |
| 4. Backface Culling | |
| 5. Rasterisation |] Programmable - Fragment Shader |
| 6. Shading - lighting + texturing | |
| 7. Hidden Surface Removal (HSR) |] Fixed (hardware) |

Object Transformation: Translation, Rotation and Scaling - Local Space $\rightarrow$ World Space.
GPUs use **SIMD** (Same Instruction, Multiple Data) to do this in parallel.

Perspective Projection: Define a virtual camera and view frustum to define what objects in the scene can be seen. The view frustum is transformed into a unit cube, transforming the coordinates from World Space $\rightarrow$ Clip Space.

Clipping: Remove anything outside the view frustum.
Note: When a triangle crosses the boundary, use algorithms such as **Sutherland-Hodgman** to only remove the parts outside of the view frustum.
Occurs after projection since calculations for a unit cube are faster than with a frustum.

Backface Culling: Remove hidden faces from objects i.e. those on the back/bottom of the object relative to the virtual camera. This is performed by calculating the dot product of the **normal vector to the face** and the **virtual camera view vector** for each face and discarding those whose dot product is > 0 .
Occurs after projection since the view vectors are all parallel in clip space.

Rasterisation: Converting continuous geometries into discrete pixels. Uses the **Scanline** algorithm to produce a **Fragment** for every pixel whose centre lies within the triangle.
— this can lead to **aliasing** (bad).

Hidden Surface Removal: GPU allocates a 2D array of floats (**z-buffer**) to store depth information for each pixel. For every fragment, compare its depth value with the one stored in the corresponding location in the z-buffer. If it is less, discard it and move on, otherwise update the z-buffer and overwrite the pixel.

Note: each fragment contains information, e.g. colour, depth, etc. needed to update a single pixel. If there are multiple fragments for a pixel (i.e. overlapping objects), then we use HSR.

Double Buffering - preventing Screen Tearing:

Instead of writing directly to the display, keep a **front buffer** and a **back buffer**.

Front Buffer: Contains the previous completed frame. Display hardware reads from this buffer.

Back Buffer: This is where the GPU draws to as it is writing each frame - not accessed by display hardware.

Once the GPU has finished drawing the next frame AND the monitor sends a **VSync** signal indicating it is ready for a new frame, the buffers are swapped; the back buffer becomes the front buffer, updating the output image on the display, and the GPU clears and begins drawing the next frame to the old front buffer.

This means that the monitor will only display complete frames $\rightarrow$ no screen tearing $\smile$

Total VRAM needed = $\text{pixels} \times (C_{\text{front}} + C_{\text{back}} + D)$

$C_{\text{front}}, C_{\text{back}}$ = Colour bytes per pixel (4B for RGBA) for front and back buffers

D = Z-depth bytes per pixel (4B for 32-bit float)

pixels = no of pixels

Transformations

Translation - Moving an object from one location to another.
- defined by a translation vector, added to the position vector to get the transformed position vector.

Rotation - Turning an object around a pivot point (usually origin)
- defined by applying trigonometry:

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

where θ is the angle of rotation.

Scaling - Resizing an object.
- defined by a scaling vector, multiplied component-wise by the position vector to get the new position.
- Can be uniform or non-uniform.

Rotation and Scaling can easily be performed using Matrix Multiplication:

$$\text{2D Rotation: } \begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

$$\text{2D Scaling: } \begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

But what about Translation?

→ need to use Homogeneous Coordinates → add a third dimension and set it to 1.
(i.e. a 2D vector $\begin{pmatrix} x \\ y \end{pmatrix}$ becomes $\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$)

Note: these are still 2D Transformations, they just use 3D vectors + matrices.

This gives us:

$$\text{Translation: } \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

$$\text{Rotation: } \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$\text{Scaling: } \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

For 3D Transformations, we add a 4th dimension set to 1 and do the same thing.

When applying a series of transformations to an object in Computer Graphics, we need to apply the same sequence of transformation matrices in the same order

(Scale → Rotate → Translate) to every vertex in the object in the GPU.

To speed up this process, we can precompute the combined transformation matrix:

$$M_{\text{combined}} = M_{\text{translate}} \cdot M_{\text{rotate}} \cdot M_{\text{scale}}$$

This is done once on the CPU, then the GPU only has to compute one matrix multiplication per vertex:

$$P' = M_{\text{combined}} \cdot P$$

Note: Transformations are applied RIGHT TO LEFT

→ in M_{combined} , the Scaling and rotation occur before translation, which is what we want.

MVP: Matrices for transformations between Model Space, World Space, View Space and Clip Space.

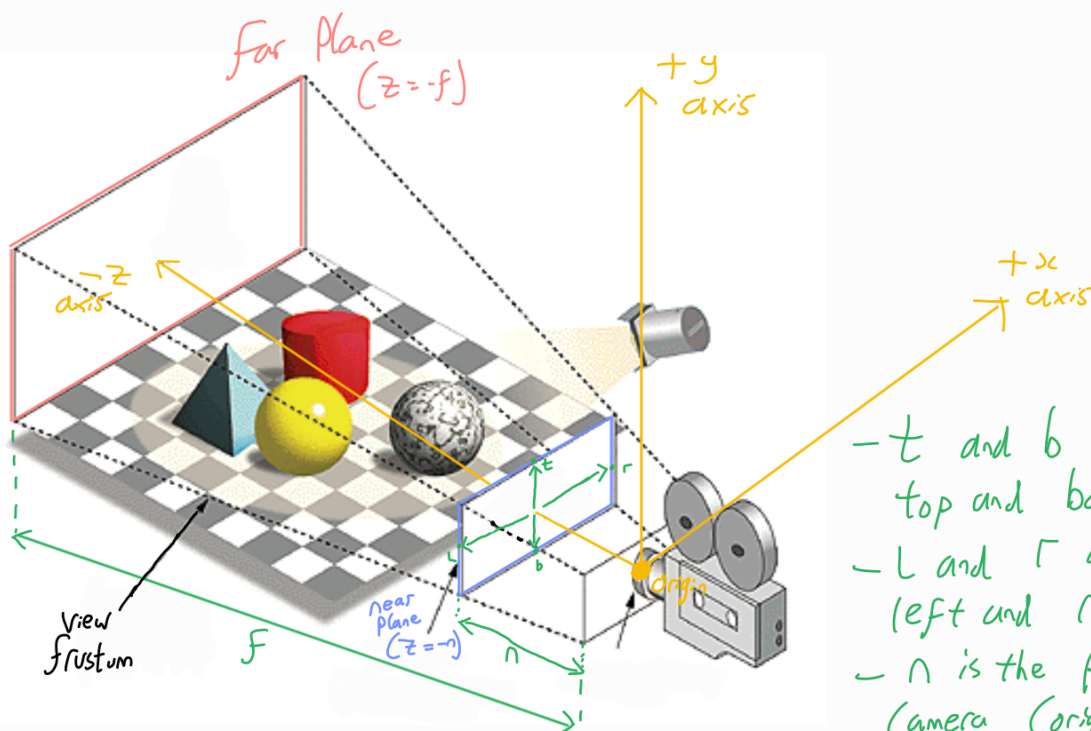
Model Matrix: This is the matrix that applies a specific sequence of scaling, rotation and translation transformations to an object to place it within world space
→ is the $M_{combined}$ from before, but in 3D (so 4×4).

View Matrix: This is the matrix that translates and rotates the entire world around a fixed virtual camera so we can see what we want to see in the scene. We transform the world coordinates from world space into view space, such that the camera is at the origin, looking down the negative Z axis.
→ This transformation consists of a translation from the world origin to the camera position followed by a rotation to align the world axes with the camera's.

Projection Matrix: Creating depth on a 2D screen.
Defines the boundaries of the world that are visible, and normalise all vertices within this area into a unit cube shape - Clip Space.
→ all visible geometries lie between -1 and +1 in clip space in the x, y and z axes → everything else is deleted.

Orthographic Projection defines a cuboidal viewing volume. In this case, a scaling and a translation operation are needed to map this to the unit cube.

Perspective Projection defines a view frustum - this requires nonlinear scaling to map to the cube:

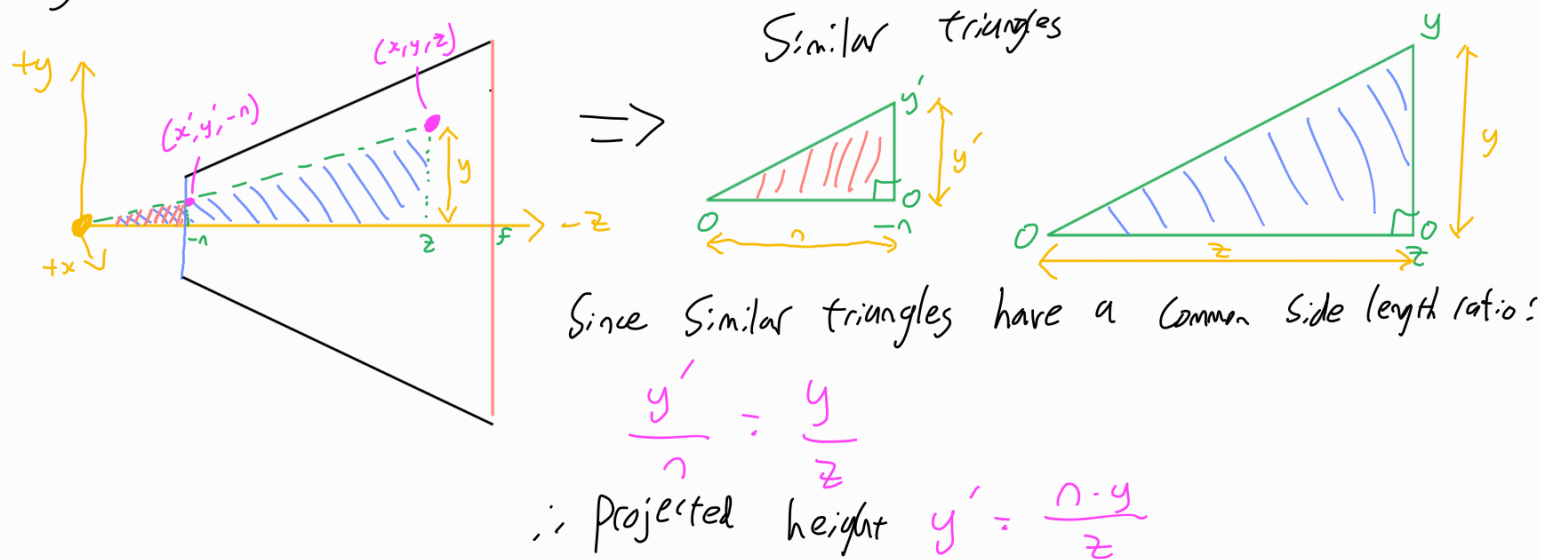


- t and b are the y-coords of the top and bottom of the near plane
- L and R are the x-coords of the left and right sides of the near plane
- n is the positive distance from the camera (origin) to the near plane
- f is the positive distance from the camera to the far plane

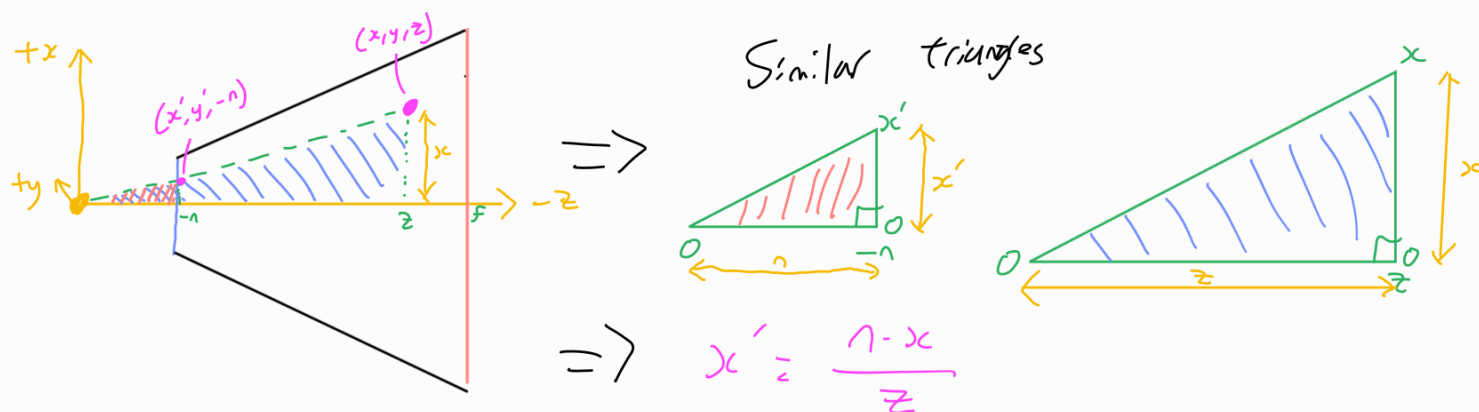
Defining L, r, t, b, n, f as in the image.

Note: L, r, t, b can be positive or negative, and usually $L = -r$ and $t = -b$
 n and f are always positive.

If we have an object within the view frustum at position (x, y, z) we need to project it onto the screen (near plane). We can do this using similar triangles:



We can do the same thing for the x -coordinate by considering a different angle:



Problem: how do we represent division by z using matrices?

$\Rightarrow$ Use the 4th component we added to homogenize the coordinates as a 'container' for the z value and let the GPU hardware deal with it.

Perspective Projection Matrix, $M_{\text{persp.}} =$

$$\begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{n-f}{f-n} & \frac{zf_n}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

This matrix transforms points from View Space into Clip Space:

$$M_{\text{persp.}} \cdot \begin{pmatrix} x_{\text{view}} \\ y_{\text{view}} \\ z_{\text{view}} \\ 1 \end{pmatrix} = \begin{pmatrix} x_{\text{clip}} \\ y_{\text{clip}} \\ z_{\text{clip}} \\ -z_{\text{view}} \end{pmatrix}$$

A **Perspective Division** stage built into GPU hardware then iterates over each vertex and normalises the vectors, producing **Normalised Device Coordinates (NDC)**:

$$P_{\text{NDC}} = \left(\frac{x_{\text{clip}}}{w_{\text{clip}}}, \frac{y_{\text{clip}}}{w_{\text{clip}}}, \frac{z_{\text{clip}}}{w_{\text{clip}}} \right) \quad (w_{\text{clip}} = -z_{\text{view}})$$

This causes objects further from the virtual camera to shrink, creating the illusion of perspective.

The three MVP matrices can then be combined into a single matrix, calculated once per object on the CPU, so that the GPU only has to perform a single matrix multiplication for each vertex making up the object.

$$M_{\text{MVP}} = M_{\text{projection}} \cdot M_{\text{view}} \cdot M_{\text{model}}$$

(Note: right to left)

Static Transformations: Calculated once and stored (e.g. projection matrices are usually constant) — computationally expensive to calculate but don't need to be calculated often.

Dynamic Transformations: Computationally cheap but need to be recalculated frequently — e.g. model and world matrices in a dynamic scene.

Lighting and Shading

Global Illumination: Simulating exact lighting physics where light rays bounce infinitely many times between surfaces. Produces photorealistic results but is extremely computationally expensive.

Local Illumination: A simplified mathematical model that only considers light rays arriving directly from the light source — No bounces.

Rasterisation pipelines render triangles one-by-one, and so when calculating lighting for an individual vertex or fragment of a triangle, it has no knowledge of other objects or triangles
 $\Rightarrow$ Local illumination is used.

To compensate for the limitations of local illumination, we use a lighting model and a shading model.

A lighting model is a set of equations used to approximate the true light physics to define what colour a point on a surface should be.

A shading model defines where and how often the lighting calculations are performed.

Blinn-Phong Lighting Model:

Breaks down lighting into three components and sums them up to get the final colour.

Ambient Component: low intensity light that illuminates an entire object a fixed colour to prevent areas in shadow from appearing pure black.

$$\text{Ambient Colour} = \text{Ambient Strength} \times \text{Light Colour} \times \text{Object Colour}$$

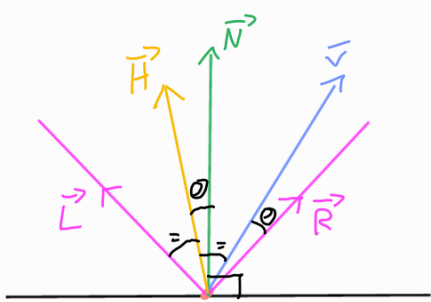
User-defined constant
ambient light colour
object albedo

Diffuse Component: light that reflects in different directions due to an object's shape. Brightness is dependent on the angle between the light direction and the surface normal (Lambert's Cosine Law) — dot product.

$$\text{Diffuse Colour} = \max(0, \frac{\mathbf{N} \cdot \mathbf{L}}{|\mathbf{N}| |\mathbf{L}|}) \times \text{Light Colour} \times \text{Object Colour}$$

prevent negative light.
Surface normal
direction vector from point on surface to light source

Specular Component: highlights caused by the viewer being perfectly aligned with light reflection. Intensity depends on the angle between the view vector and the reflection vector, with smaller angles = brighter intensities.



Optimisation: Computing the reflection vector $\vec{R}$ is difficult so we use the angle between the halfway vector $\vec{H}$, which is half way between the incoming light vector $\vec{L}$ and the view vector $\vec{V}$.

This optimisation differentiates Blinn-Phong lighting from pure Phong lighting.

$$\text{Specular Colour} = (\max(0, \vec{N} \cdot \vec{H}))^\alpha \times \text{Specular Strength} \times \text{Light Colour}$$

User-defined constants

Final Blinn-Phong Equation:

$$I_{\text{final}} = (k_a I_a) + (k_d (\vec{N} \cdot \vec{L}) I_l) + (k_s (\vec{N} \cdot \vec{H})^\alpha I_l)$$

- I_{final} : The final light intensity.
- k_a, k_d, k_s : Ambient, diffuse, and specular coefficients (strengths).
- I_a, I_l : Ambient and light source intensities (colours).
- $\vec{N}, \vec{L}, \vec{H}$: Normal, Light, and Halfway vectors.
- α : Shininess exponent.

This resulting light colour is then multiplied by the object's albedo (base colour) to get the final fragment colour.

Note: Lighting calculations depend heavily on the surface normal vector. As a result, it is essential that the stored value of this vector remains accurate when objects are transformed (eg by non-uniform scaling) otherwise lighting will break.

To rectify this, when applying a transformation to an object using a model matrix M , calculate the Normal matrix $(M^{-1})^T$. This is done on the CPU and passed to the vertex shader, which is responsible for correctly transforming the vertex normals (as well as the vertex positions as a result of M).

$$\vec{N}_{\text{Transformed}} = (M^{-1})^T \vec{N}_{\text{orig}}$$

Types of Light Sources:

Directional Light: a light source treated as being infinitely far away, with parallel light rays, i.e. the sun (deadly laser)

- The light vector $\vec{L}$ is constant, and there is no attenuation (change in intensity over distance)

Point Light: a light source with a specific position in the world which emits light in all directions, e.g. a light bulb.

- $\vec{L}$ is different for each fragment: $\vec{L} = P_{\text{light}} - P_{\text{frag}}$
- Attenuation - light intensity diminishes with distance.

$$\text{Attenuation} = \frac{1}{k_c + k_l \cdot d + k_q \cdot d^2}$$

Where K_c, K_L, K_q are user-defined Constant, linear and Quadratic coefficients and d is distance.

- Implementing point lights requires the Vertex Shader to calculate the vertex positions in World Space (NOT Clip/Screen space) and pass this to the Fragment Shader, which uses it to calculate the light vector $\vec{L}$ and the distance from the light source.

Spotlights: an extension of Point lights, only light is not emitted in all directions.

Shading Models:

Attribute	Flat Shading	Gouraud Shading	Phong Shading
Calculation Point	Per-Triangle	Per-Vertex	Per-Fragment
Quality	Low (Faceted)	Medium (Smooth)	High (Very Smooth)
Performance Cost	Very Low	Low	High
Specular Highlights	Not possible	Poor / Inaccurate	Excellent / Accurate
Modern Standard	No (Artistic choice)	No (Largely obsolete)	Yes

Geometry and Texture Mapping

Meshes

A **mesh** is the collection of vertices, edges and faces that make up an object, defined by two lists of data:

Vertices - list of all unique points making up the shape, stored on the GPU in a **Vertex Buffer Object (VBO)**

Each vertex is a collection of attributes, including:

- Position (x, y, z)
- Normal vector of the surface at that vertex.
- texture coordinates (u, v)
- Colour

Indices - list of integers corresponding to the indexes of vertices in the **vertices list**. The order of the indexes in this list provides instructions for connecting the vertices to form triangles. Stored on the GPU in an **Index Buffer Object (IBO)**

We use an Index Buffer to avoid storing the full vertex information multiple times for vertices which are shared between multiple triangles



Texture Mapping

Texture mapping allows us to wrap a 2D texture onto the surface of a 3D object. Pixels in the texture image are known as **texels** for some reason.

Forward Mapping - iterating over every **texel** and mapping it to a point on the 3D geometry.
- bad as it leaves visual artefacts, e.g. if the texture resolution is smaller than the 3D object

Backward (inverse) mapping - iterating over every **on-screen pixel of the 3D geometry** and determining which **texel** should be mapped to it.
- No holes since every on-screen pixel is guaranteed to be mapped to a texel.

To control how a texture maps to a 3D surface, we use **UV unwrapping** to unfold the 3D mesh into 2D space, with **UV coordinates** assigned to each vertex. These are stored as vertex attributes and passed from the Vertex Shader to the Fragment Shader. During Rasterisation the GPU interpolates these attributes across all fragments in the triangle.

Note: **UV coordinates** are normalised to $[0, 1]$, with $(0, 0)$ being the bottom left and $(1, 1)$ being top right.

Mip-Mapping - pre-calculating a sequence of lower-resolution versions of a texture and mapping the most appropriate version based on distance to the object - different levels of detail. This helps improve performance and decreases aliasing in areas with high-freq. details.

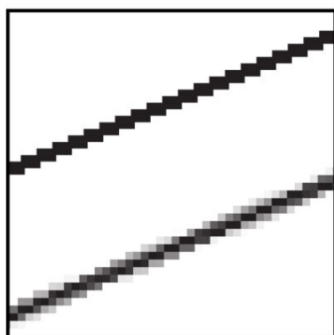
Anti-Aliasing

Aliasing is the problem where the edges of shapes appear jagged, due to limitations of the rasterisation process in representing continuous images using discrete pixels.

WITHOUT ANTI-ALIASING



WITH ANTI-ALIASING



Anti-aliasing Methods:

Supersampling (SSAA):

Render the entire scene at a much higher resolution (e.g. 4x screen resolution) and take the average of e.g. the four high-res pixels per screen pixel. Significantly computationally expensive.

Multisampling (MSAA):

don't need to know what stencil does (?)

Stores Depth and Stencil values at multiple sub-sample locations within each pixel. If a triangle only covers some of the subsamples, the hardware will mix the calculated colour within the fragment with the background colour based on the percentage of subsamples covered.

This only smooths geometric edges, not aliasing present in textures or shading effects.

Fast Approximate Anti Aliasing (FXAA):

Performs post-processing on the fully rendered frame. This involves searching for high-contrast edges and applying blur filters to smoothen them. Extremely fast and efficient, works on all types of aliasing (unlike MSAA), but may create unwanted blurring of fine texture details.

Texturing Techniques

Billboarding: representing a 3D object using a 2D quad (2 triangles) programmed to rotate to always face the virtual camera.

Normal Mapping: instead of rendering all the details of an object in the geometry, use a map of 'fake' normal vectors in the lighting calculations to make flat surfaces appear detailed.

Environment Mapping: Treat the background environment as a 'cube map' surrounding the scene on all sides. When rendering a reflective surface, calculate the reflection vector and see where it intersects the cube map, and use the colour at that location to calculate the colour of that pixel on the reflective surface.

Scene Graph

The **Scene Graph** is a Directed, Acyclic graph where each node represents an object (3D model, light source etc.) in a hierarchical structure.

Each node stores its own **local transformation** (relative to its parent node in the graph). The final **world transformation** of the node is calculated by combining the **parent's world transformation** with this **local transformation**:

$$M_{\text{world}} = M_{\text{parent-world}} \times M_{\text{local}}$$

The final world transformation matrix (M_{world}) is the Model Matrix used to produce the MVP matrix, which is sent to the Vertex Shader in the Transformation Step of the rendering pipeline.

Note: the CPU handles all the scene graph hierarchy stuff. The GPU (vertex Shader) only receives the MVP matrix for each object and performs the matrix multiplications.

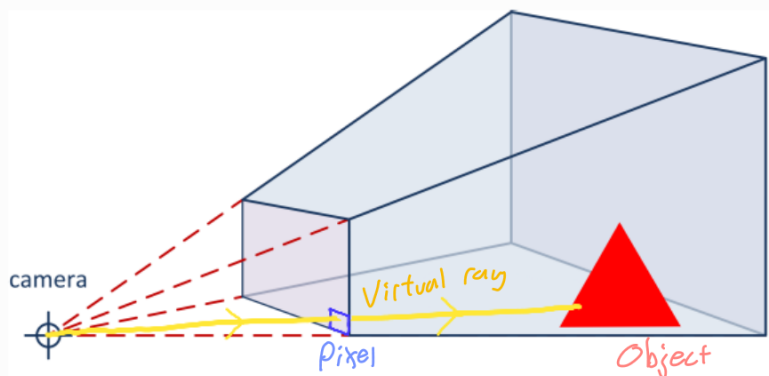
Applications:

Application	Concept & Role	Key Benefit
Construction	Building complex objects from simpler parts (e.g., Car Chassis + Wheels).	Relative positioning simplifies modelling.
Animation	Forward Kinematics: Moving a parent node automatically moves all children.	Complex motion (like a walking character) becomes manageable.
Optimisation	Frustum Culling: Using Bounding Volume Hierarchies (BVH) to skip invisible branches.	Reduces checks from linear $O(N)$ to logarithmic $O(\log N)$.
Management	Organizing entities into logical groups (e.g., "Lights", "NPCs").	Enables batch operations (toggling visibility, filtering).

Ray Tracing

Rasterisation takes 3D models made up of triangles and projects them onto a 2D screen, filling in discrete pixels covered by these triangles. It is an example of **object-order rendering**, where each triangle is iterated through to produce each frame.

Ray-Tracing is an example of **image-order rendering** - it iterates through each pixel of the final image, casting a virtual ray of light from the virtual camera through this pixel to the scene. The pixel colour is determined by the properties of the first surface the ray hits.



result: pixel is red.
repeat for all pixels.

The virtual rays are defined by their starting point O and direction vector $\vec{D}$. Any point $P(t)$ along a ray can be described by the parametric equation $P(t) = O + t\vec{D}$. Note: $\vec{D}$ is normalised. The parameter t represents the scalar distance of $P(t)$ from O .

To calculate the point of intersection between a ray and an object, substitute the ray equation into the equation for the surface of the object.

For example, the equation $(P-C) \cdot (P-C) = r^2$ represents all points P on the surface of a sphere of radius r and centre C . Substitute $P = O + t\vec{D}$ and solve the quadratic for t . If there are no solutions, the ray does not pass through the sphere, if there are multiple solutions take the one with the smallest positive value of t .

Rays travelling from the camera into the scene are **primary rays**. If a ray hits a reflective surface, we calculate the direction of reflection and recursively cast new rays until a non-reflective surface is found or a specified recursion limit is reached (to prevent infinite reflection). After finding the intersection point between the ray and a surface we cast **shadow rays** from this point to all light sources. If a shadow ray to a light source is obstructed by another object, the original point is in shadow from this light and will not receive any diffuse or specular illumination from that particular light source.

The final colour of a pixel is calculated by combining **local illumination** from light sources (modulated by **shadow rays**) and the **reflected illumination** (calculated from the recursive reflection ray).

Note: We still use Blinn-Phong lighting with ray tracing (as with rasterisation).